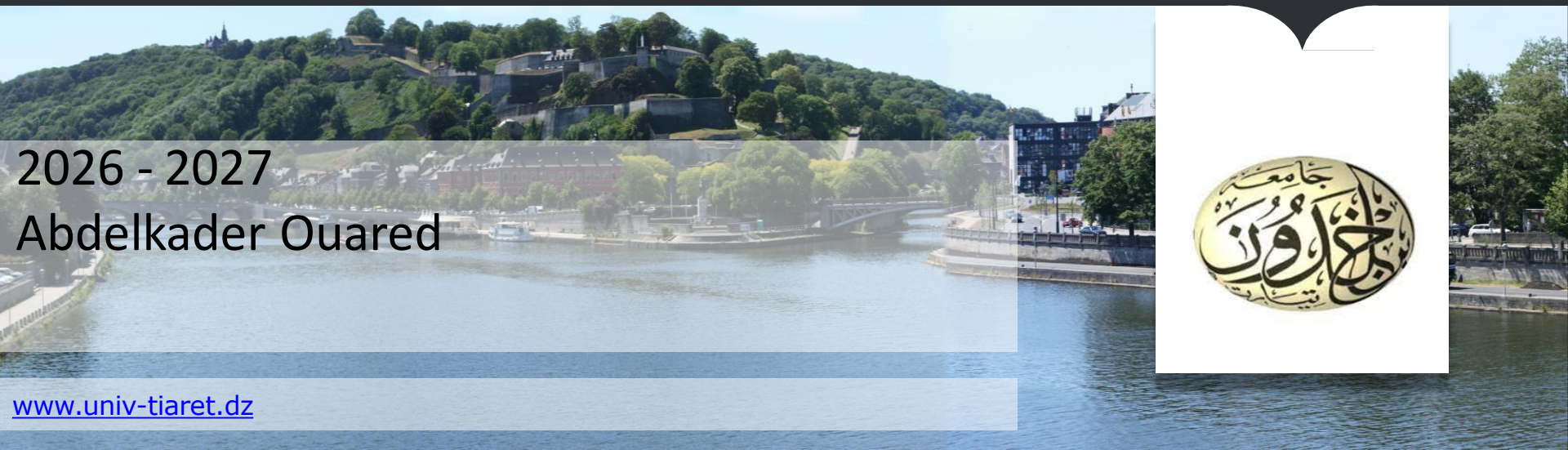
Programmation Mobile

Anatomie d'une Application Android



2026 - 2027

Abdelkader Ouared

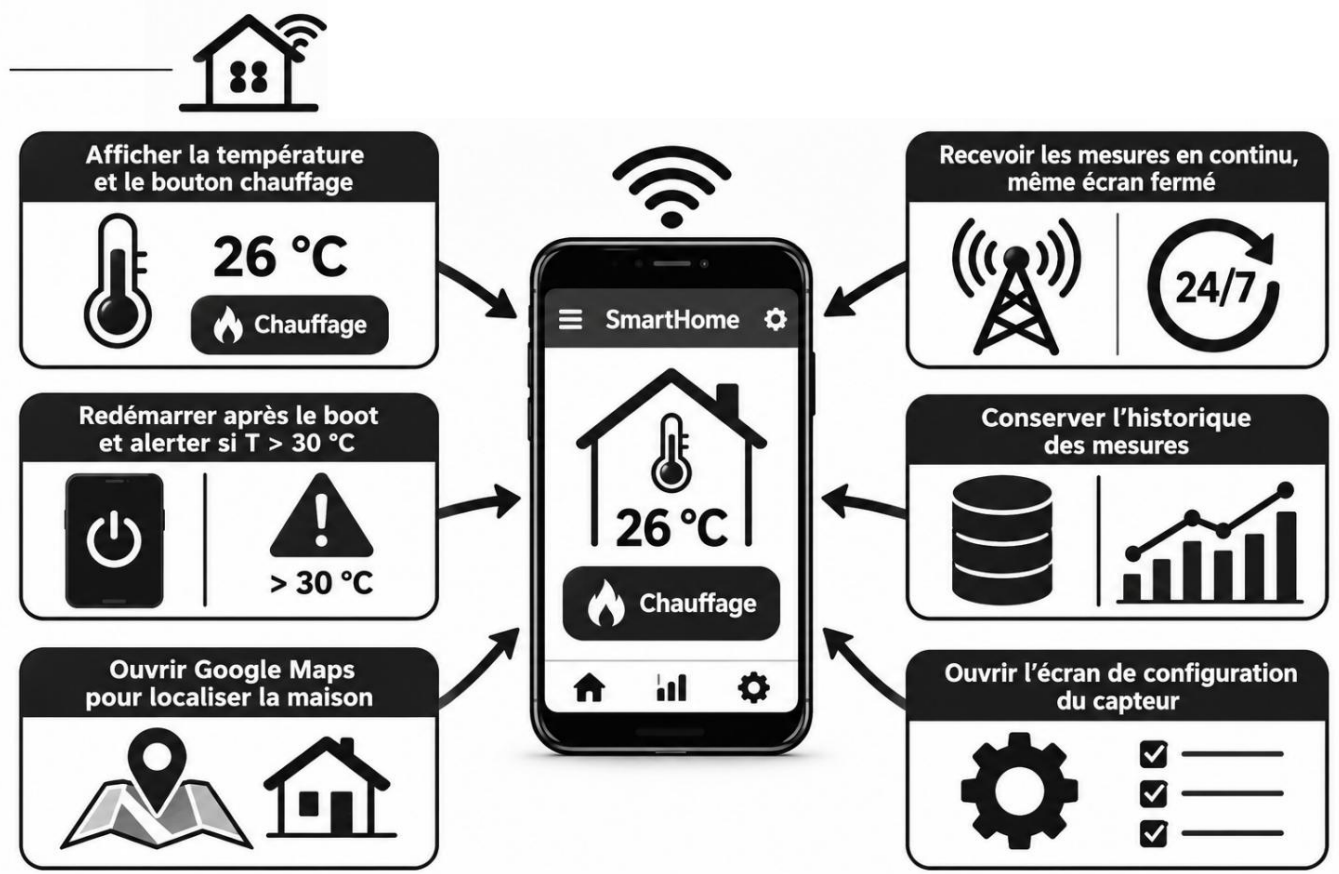
Motivation



Application « SmartHome »

Scénario

Une application Android contrôle un capteur de température IoT (ESP32) installé dans une maison.



Application « SmartHome »

Scénario

Une application Android contrôle un capteur de température IoT (ESP32) installé dans une maison. Elle doit répondre aux 4 besoins du tableau ci-dessous.

Besoin	Composant Android	Pourquoi ?
1. Afficher la température et le bouton chauffage.		
2. Recevoir les mesures en continu, même écran fermé.		
3. Redémarrer l'application après le boot du téléphone et alerter si la température dépasse 30 °C.		
4. Conserver l'historique des mesures, lisible par d'autres applications.		
5. Ouvrir Google Maps pour localiser la maison.		
6. Ouvrir l'écran de configuration du capteur depuis l'écran principal.		

Application « SmartHome »

Scénario

Une application Android contrôle un capteur de température IoT (ESP32) installé dans une maison. Elle doit répondre aux 4 besoins du tableau ci-dessous.

Besoin	Composant Android	Pourquoi ?
1. Afficher la température et le bouton chauffage.		
2. Recevoir les mesures en continu, même écran fermé.		
3. Redémarrer l'application après le boot du téléphone et alerter si la température dépasse 30 °C.		
4. Conserver l'historique des mesures, lisible par d'autres applications.		
5. Ouvrir Google Maps pour localiser la maison.		
6. Ouvrir l'écran de configuration du capteur depuis l'écran principal.		

Motivation



Introduction à Android

Android

Android SDK

Android Studio

Introduction au système d'exploitation Android

- **Qu'est ce qu'Android**
- Android est une pile logicielle (**Software Stack**) destinée aux smartphones et aux tablettes, développée et proposée par Google. Elle comprend notamment :

  - Un OS (Operating System) basé sur le noyau Linux.
  - Un ensemble de drivers pour piloter les composants matériels (appareils photos, WIFI, écran tactile, GPS,..).
  - Un ensemble de libraires accessibles via le langage de programmation Java/Kotlin.
  - Un ensemble d'applications basées ces libraires.

Introduction au système d'exploitation Android

Smartphone / Tablette



Android est une pile logicielle complète : noyau Linux, drivers, bibliothèques, Android Runtime (ART), framework et applications.

Smartwatch



Certaines montres utilisent **Wear OS**, un système d'exploitation basé sur Android.

Réalité virtuelle (VR)





Certains casques utilisent un système basé sur Android (par ex. **Meta Horizon OS**).

Introduction au système d'exploitation Android

- La start-up avait initialement pour objectif de développer des systèmes d'exploitation pour les caméras numérique
- Le nom Android vient du nom de l'entreprise qui a initialement crée le système d'exploitation, avant son achat par Google
- Entièrement **personnalisable** sur le noyau Linux

Android Inc. a été fondée en **octobre 2003** par Andy Rubin, Nick Sears, Chris White et Rich Miner

Projet Android Open Source: https://source.android.com/?hl=fr

Android SDK (Software Development Kit)

- **Android SDK (Software Development Kit) :** Android SDK est un ensemble d'outils de développement qui permet aux développeurs de créer des applications Android.
- Il comprend divers composants, tels que **les bibliothèques** de code, les **émulateurs**, les **outils de débogage**, les **bibliothèques tierces**, et, les **API Android**.

- **Android SDK:** développer, simuler, tester et analyser des applications Android, sans nécessairement disposer d'un appareil physique.

- L'Android SDK peut être utilisé avec n'importe quel environnement de développement.

Android Studio

- **Android Studio :** est l'environnement de développement officiellement recommandé par Google pour le développement d'applications Android. C'est un IDE (Integrated Development Environment) basé sur IntelliJ IDEA et spécialement conçu pour Android.

- Android Studio ne sert pas uniquement à écrire du code: il permet également de *visualiser*, *déboguer*, *optimiser et analyser les applications Android.*

Outils d'analyse intégrés à Android Studio

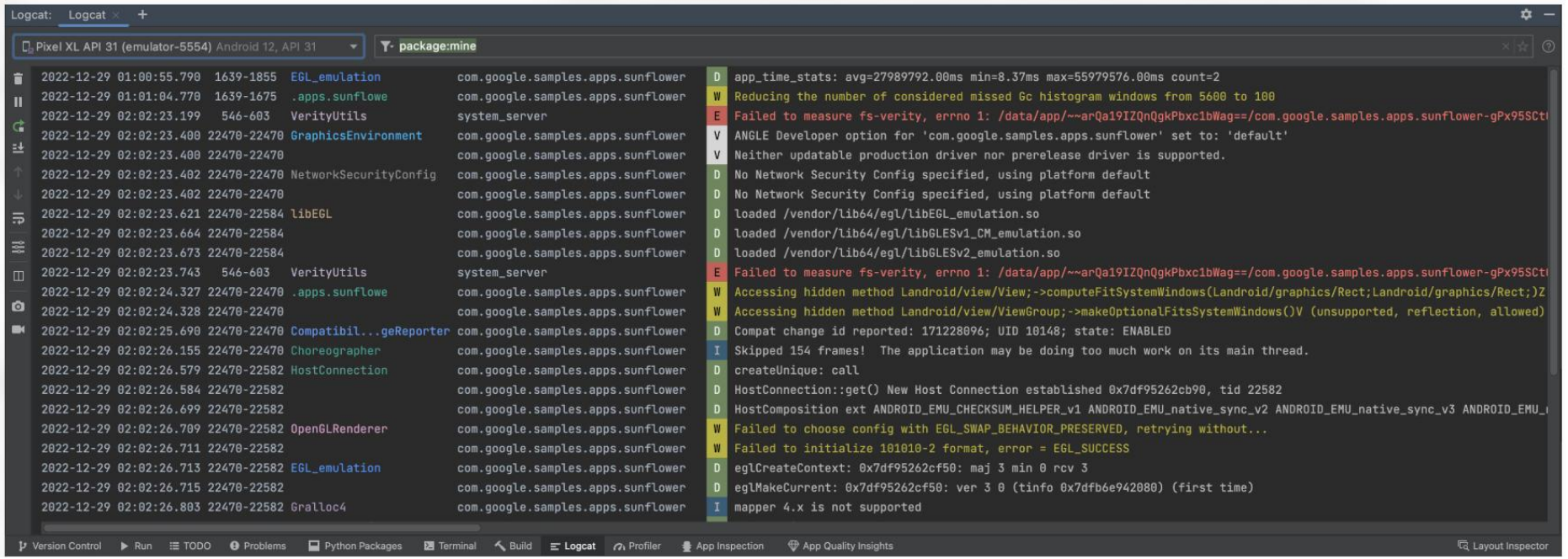
Logcat Profiler Lint Gemini (AI Assistant)

Android Studio

Outils d'analyse intégrés à Android Studio

Logcat

Que se passe-t-il dans l'application ?

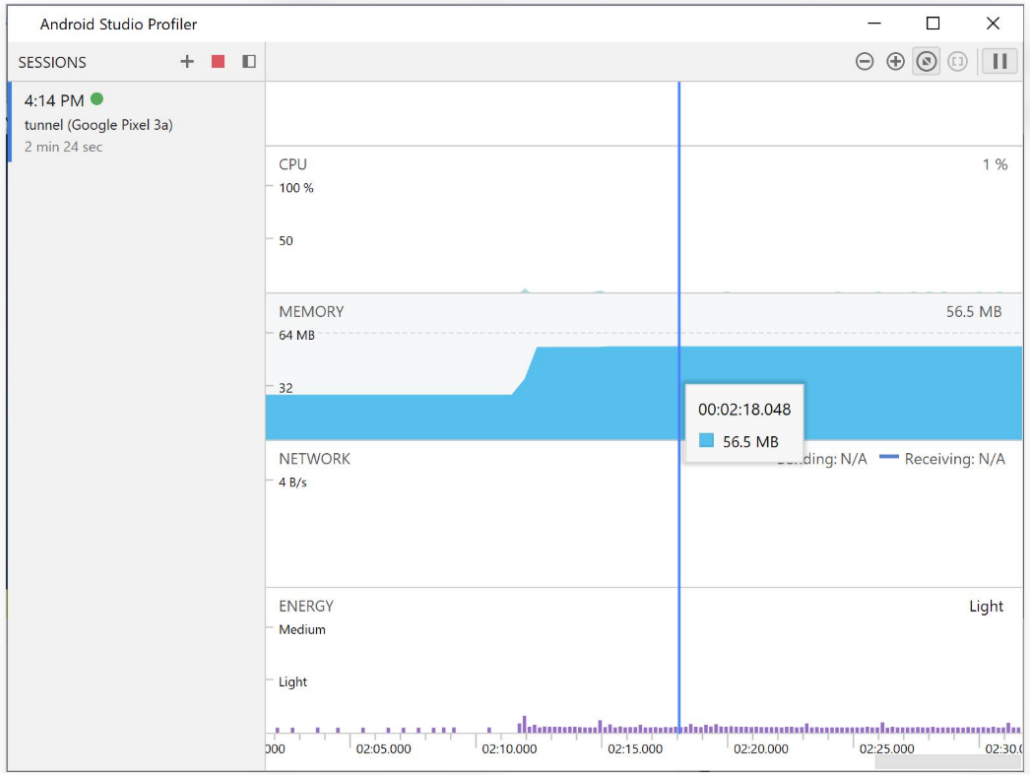


Android Studio

Outils d'analyse intégrés à Android Studio

Profiler

Combien de ressources consomme-t-elle ?

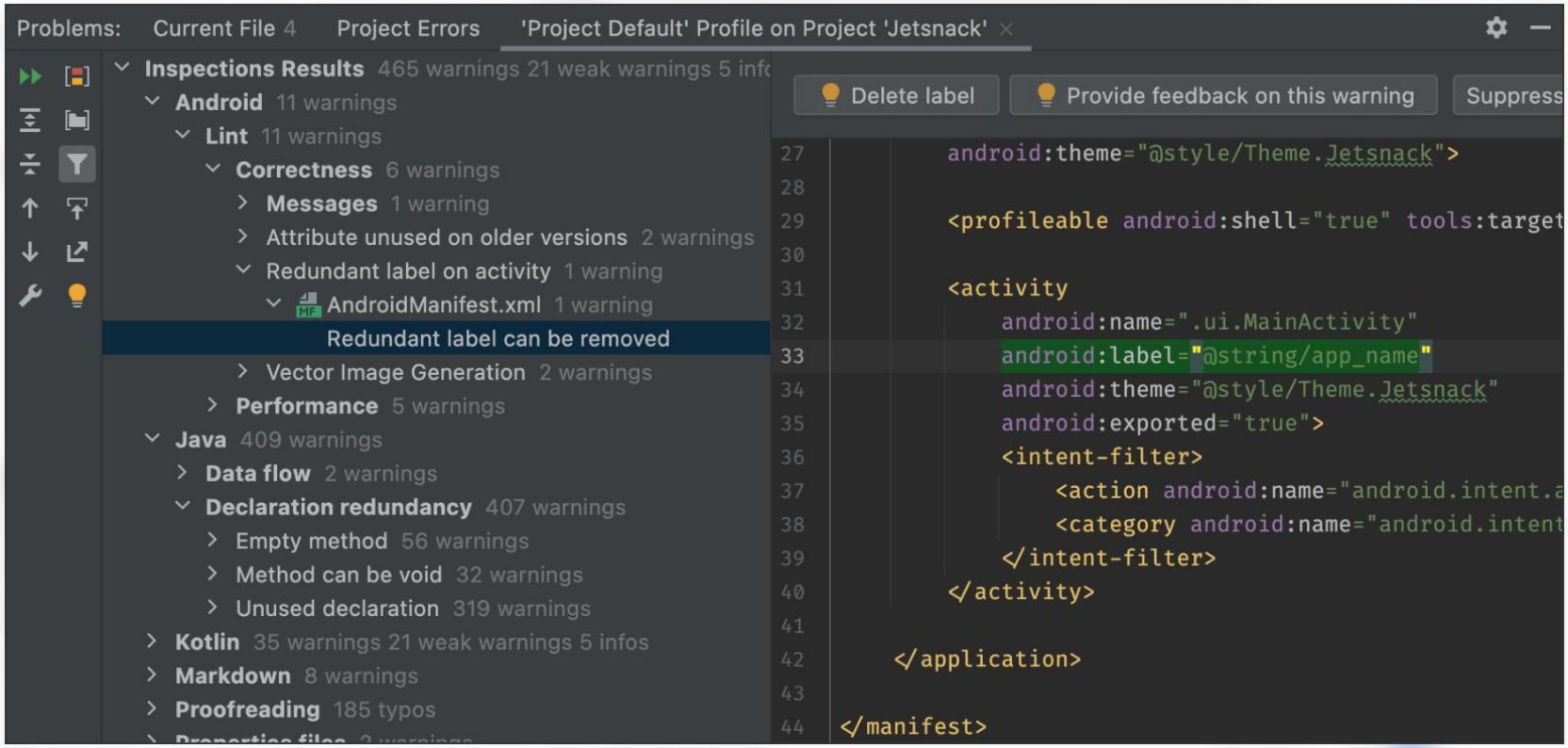


Android Studio

Outils d'analyse intégrés à Android Studio

Lint

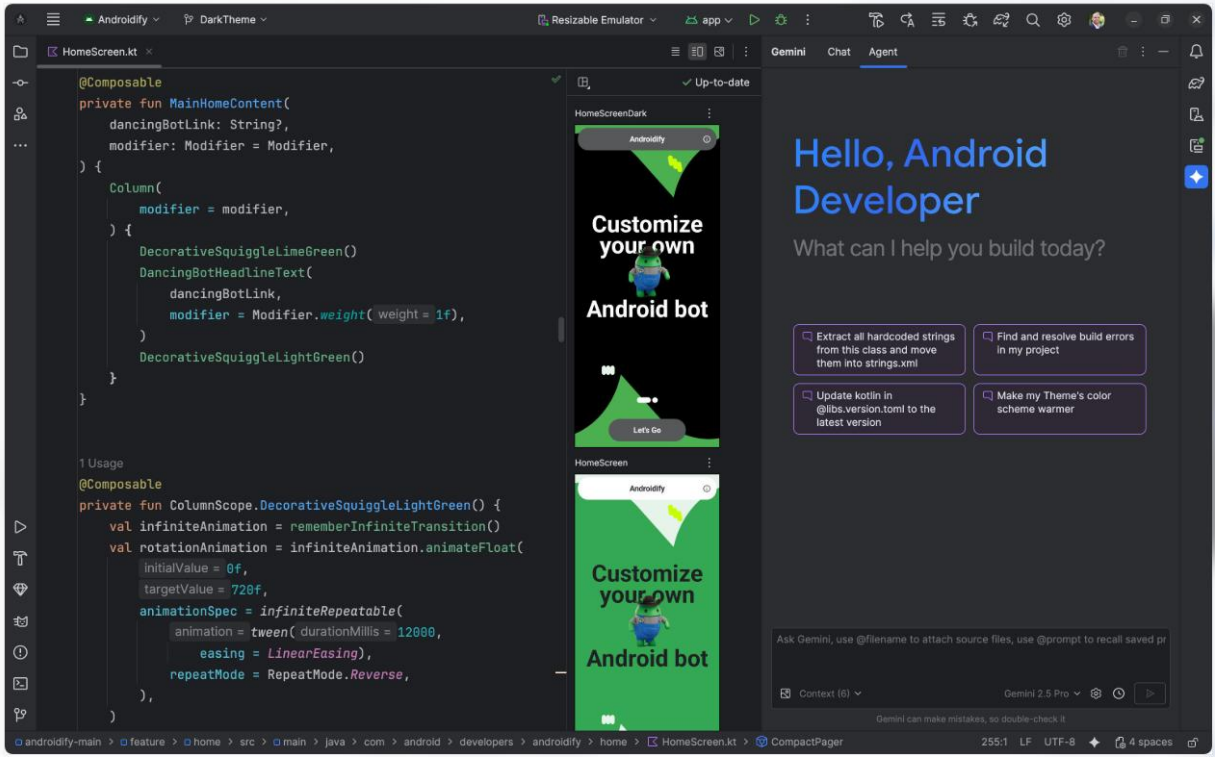
Quels problèmes existe-t-il dans le code ?



Android Studio

Outils d'analyse intégrés à Android Studio

Gemini (AI Assistant)
obtenir une assistance par IA pour générer, expliquer et améliorer le code.



Exercice: Développement Android assisté par IA

1. Développer: Créer une application de gestion des tâches avec Kotlin. (Android Studio)

2. Versionner le projet: Créer un dépôt GitHub, effectuer des commits et suivre les modifications du code.

2. Générer avec l'IA: Générer et modifier le code source (OpenCode)

3. Tester et corriger: Exécuter l'application, réaliser les tests et corriger les erreurs. (Android Studio / Lint)

4. Analyser le code: Détecter bugs, défauts de conception et problèmes de qualité (OpenCode Skills)

- https://github.com/obra/superpowers/blob/main/skills/brainstorming/SKILL.md

À retenir…

- **Android** : Système d’exploitation mobile basé sur le noyau Linux.

- **Android SDK (Software Development Kit)** : Ensemble d’outils et de bibliothèques permettant de développer des applications Android.

- **Android Studio** : Environnement de développement intégré (IDE) officiel pour créer, tester et déboguer des applications Android.

ARCHITECTURE

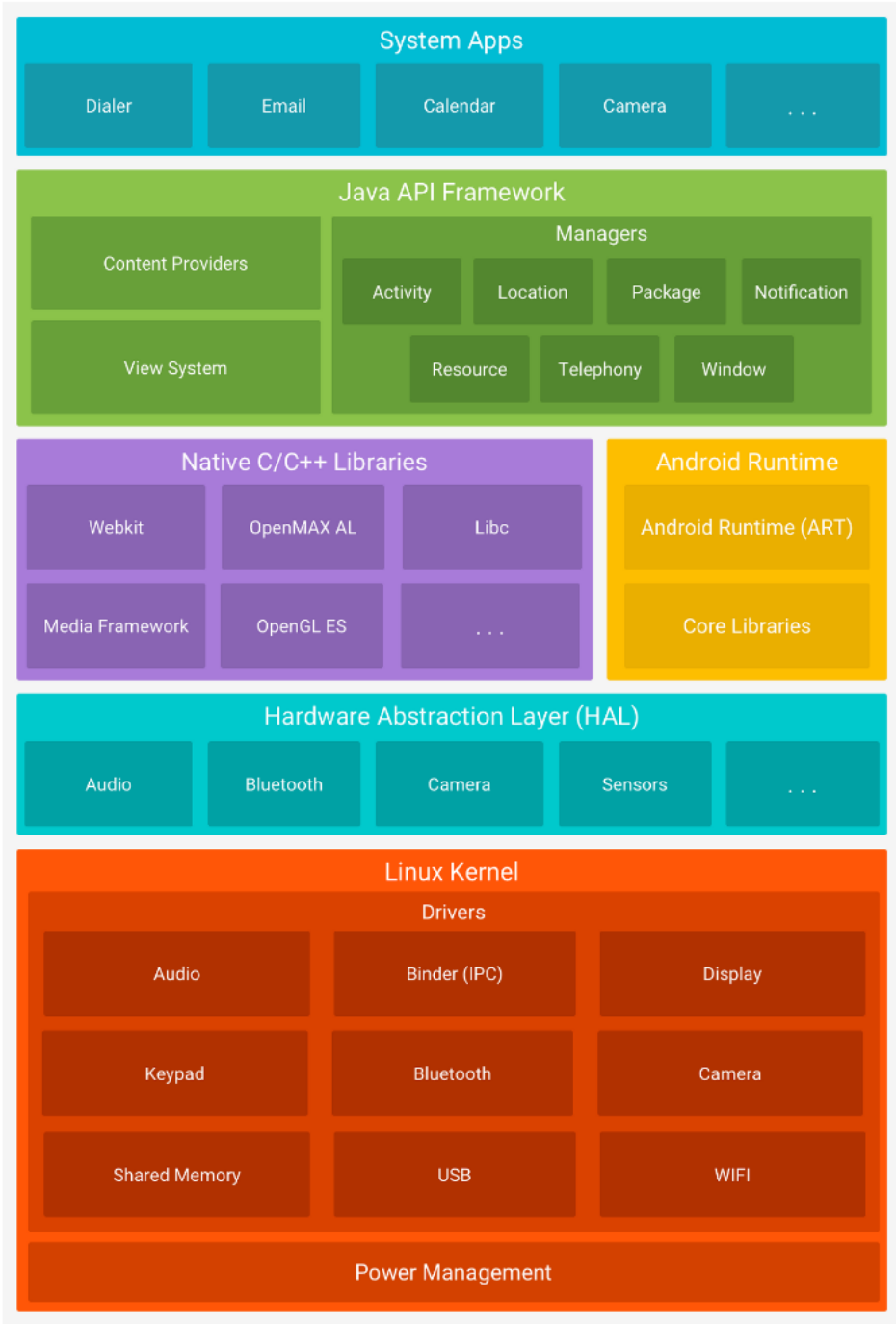


Architecture des applications Android

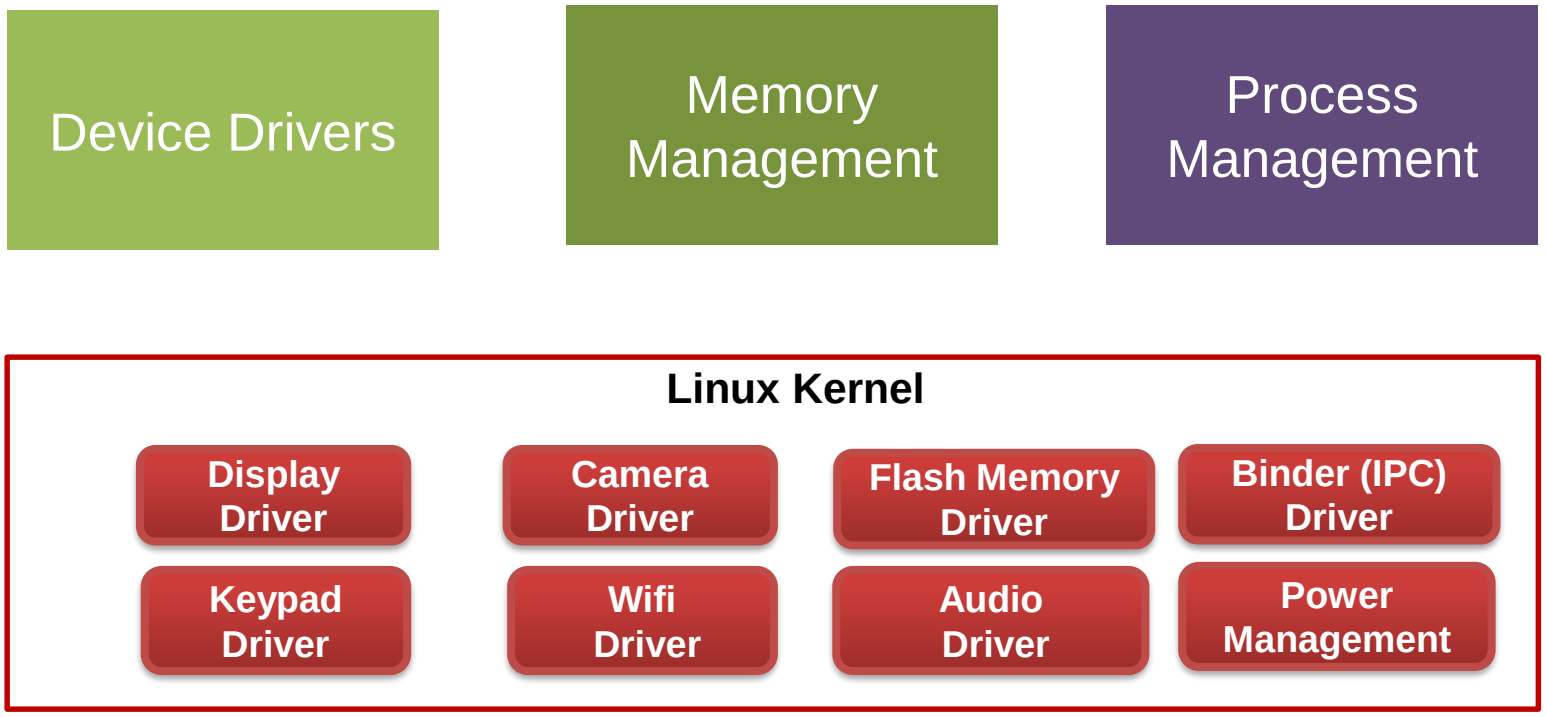
Google avait publié le code Android en open source sous **licence Apache**

Peut être modifié, si nécessaire
Par ex, Facebook et X proposent leurs propres versions.

Pas de redevances ni de frais de licence

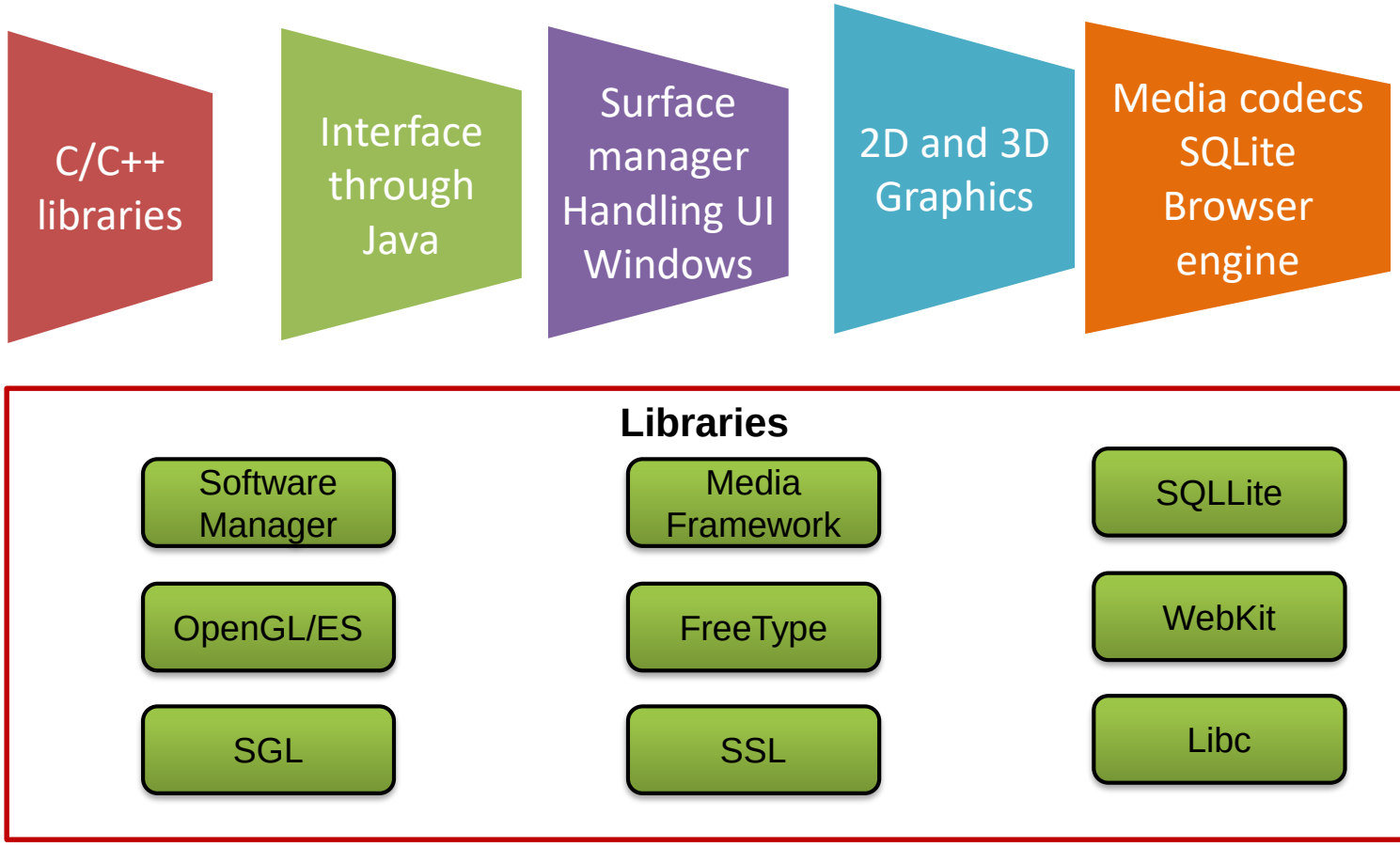


Architecture des applications Android: Linux Kernel



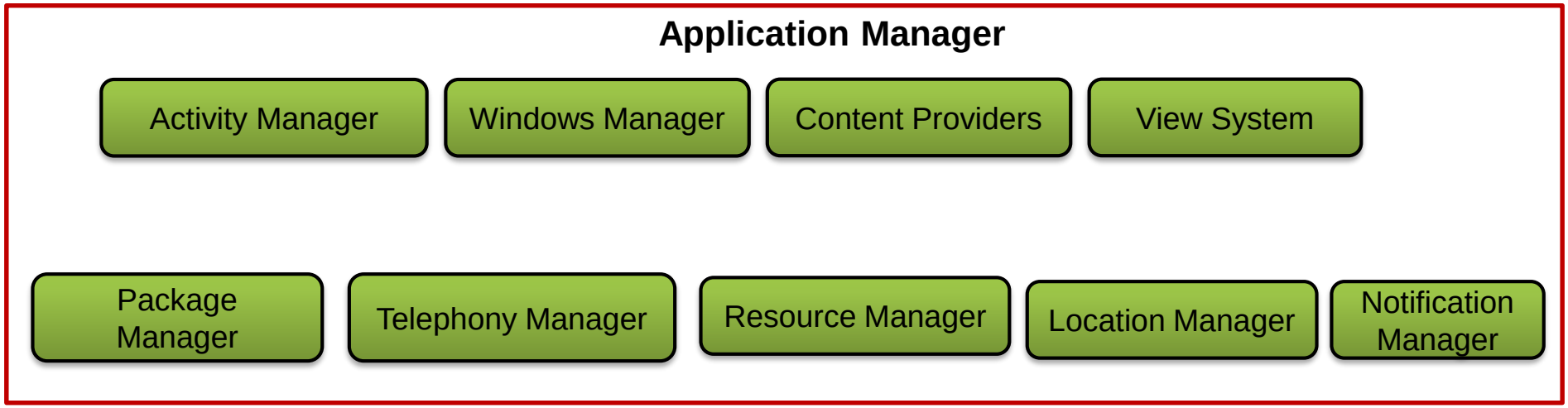
Ex. : Accéder à la liste des réseaux Wi-Fi nécessite beaucoup de code sans les frameworks Android.
Linux Kernel : principalement développé en C et en assembleur.

Architecture des applications Android: Libraries



Architecture des applications Android: Application Manager

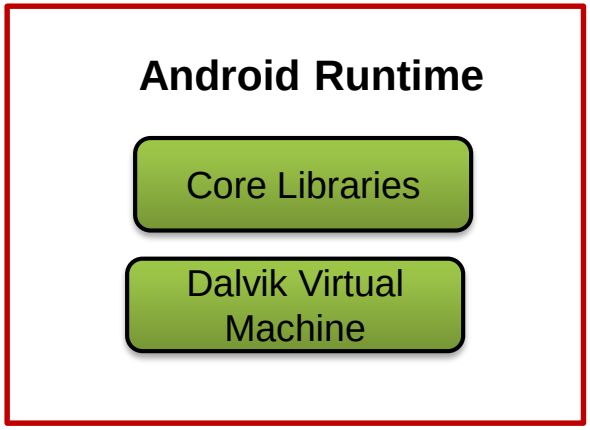
- **Interface API**
- **Gestionnaire d'activités** : gère le cycle de vie des applications



Architecture des applications Android: Android Runtime (ART)

ART: Moteur d'éxécution des applications Android directement sur l'appareil

- **Dalvik VM**
- Fichiers compilés Dex (Dalvik Executable) spécifiques à l'environnement d'exécution Android
- Compact et efficace par rapport aux fichiers de classe en termes de mémoire et de puissance de la batterie.
- **Core Libraries (libcore)**
- un ensemble de bibliothèques Java incluses dans le runtime d'Android
  ex. java.lang, java.util, java.io

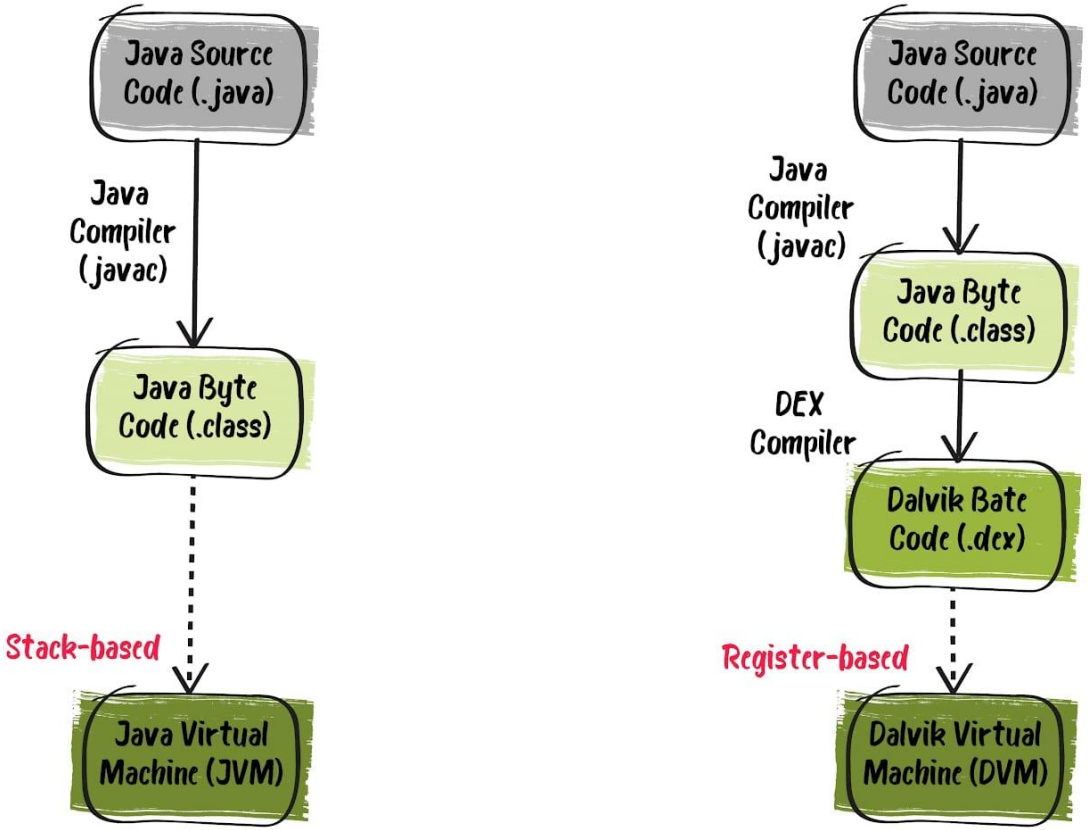


Architecture des applications Android: Android Runtime

Dalvik virtual machine: JVM optimisé pour les platformes mobiles

- **Dalvik n'est pas compatible Java SE**
  - Machine à processeurs (et non à pile, comme une JVM traditionelle)
- **.java => .class => .dex**
  - .dex = fichier en langage machine natif pour la VM Dalvik
  - .dex produit par le post-compilateur dx.
- **Dalvik est en interconnection directe avec le noyau Linux**

Architecture des applications Android:
Android Runtime



JVM vs DVM

COMPOSANTES D'UNE APPLICATION ANDROID



Composante de l’application

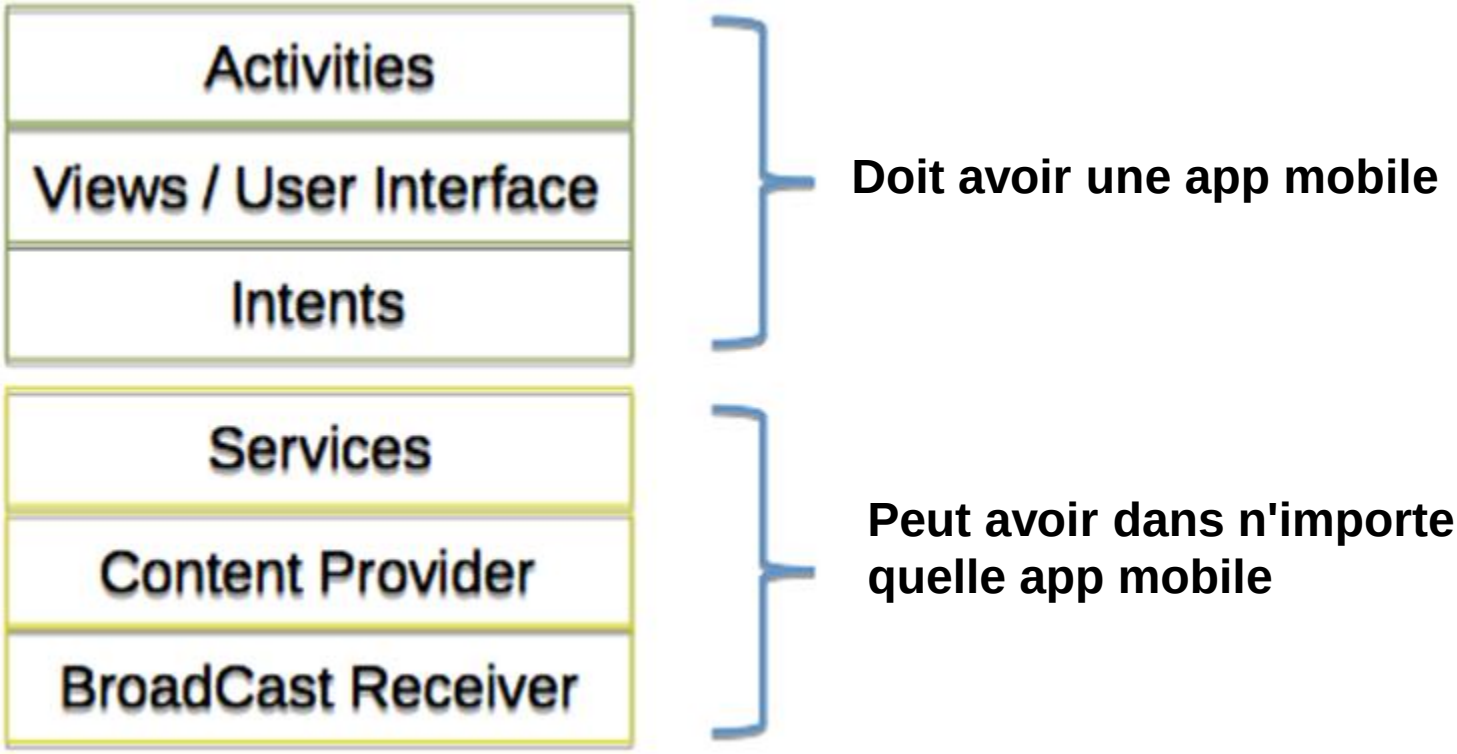
Composants de base

- Activities
- Services
- Broadcast Receivers
- Content Providers

Composants supplémentaires

- Fragements
- Views
- Layouts
- Resources
- Manifest

Composante de l’application

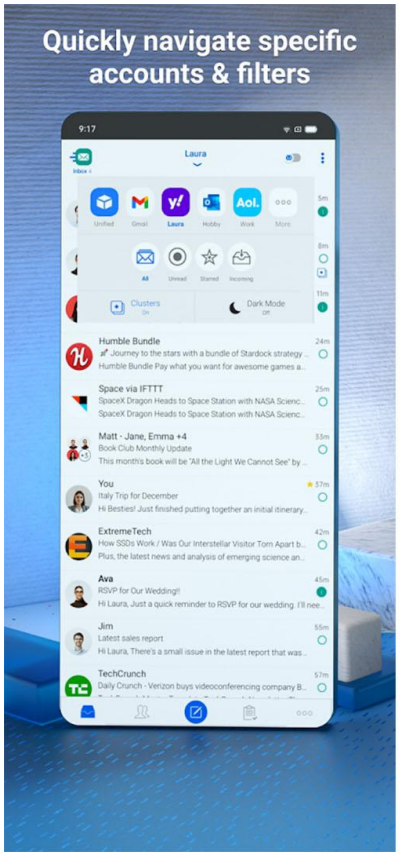


Activity

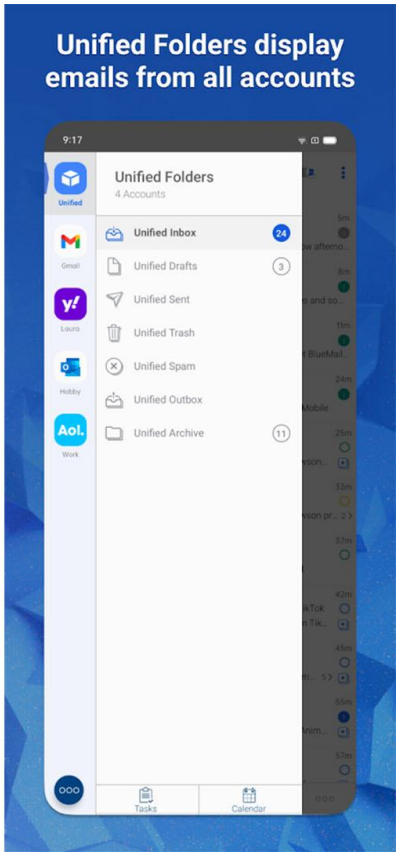
- Représenter un seul écran avec une interface utilisateur
- Contient une ou plusieurs vues (**View**: Composants de l'interface utilisateur)
- Gérer les interactions des utilisateurs
- Chaque activité est complètement isolée les unes des autres
- Une application différente peut invoquer l'activité d'une autre application, si cela est autorisé

Activity

- **Exemple : Application Email**







Activity

- **Implementation**

java

```java
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Your code for initializing the activity goes here
    }
}
```

kotlin

```kotlin
class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Your code for initializing the activity goes here
    }
}
```

Services

- Est un composant qui gère l'application associée au traitement en arrière-plan
- Opérations de longue durée
- Aucune interface utilisateur
- Exécution sans blocage de l'interaction de l'utilisateur

- **Exemples:**
  - Service de Téléchargement
  - Service de Lecteur de Musique
  - Service de Sauvegarde Automatique
  - Service de Mise à Jour des Données

Services

- **Implémentation**

java

```java
public class MyService extends Service {
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // Perform long-running operation here
        // When the task is complete, stop the service using stopSelf()
        stopSelf();
        return START_STICKY;
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

Services

- **Implémentation**

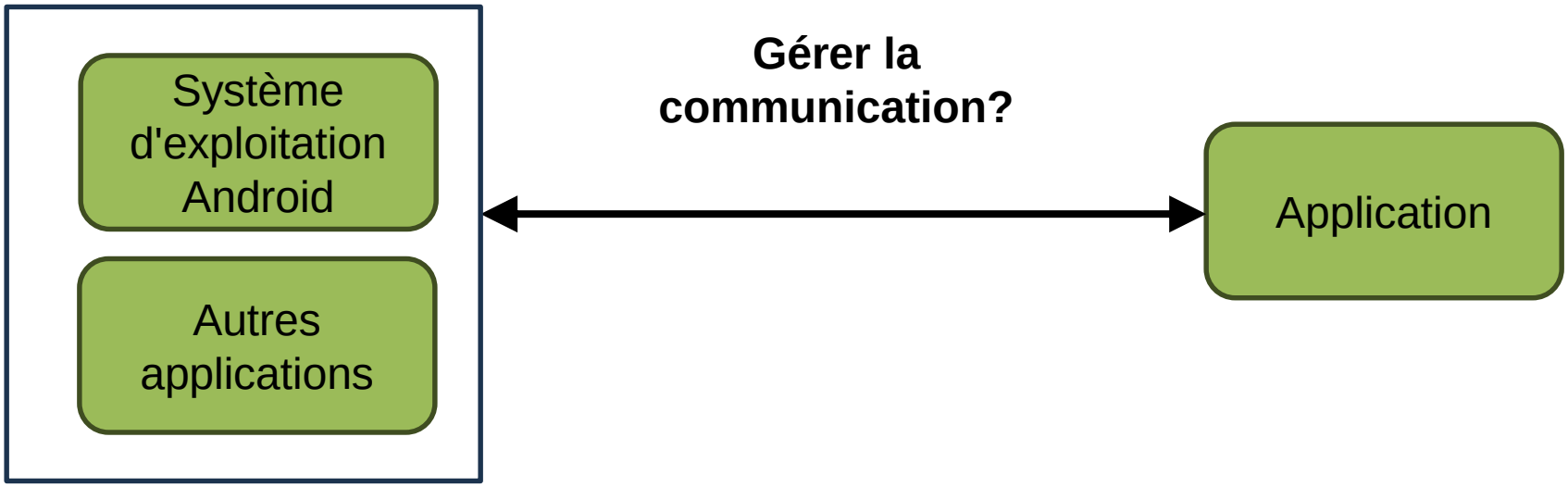
kotlin

```kotlin
class MyService : Service() {

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int):
        // Perform long-running operation here
        // When the task is complete, stop the service using stopSelf()
        stopSelf()
        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder? {
        return null
    }
}
```

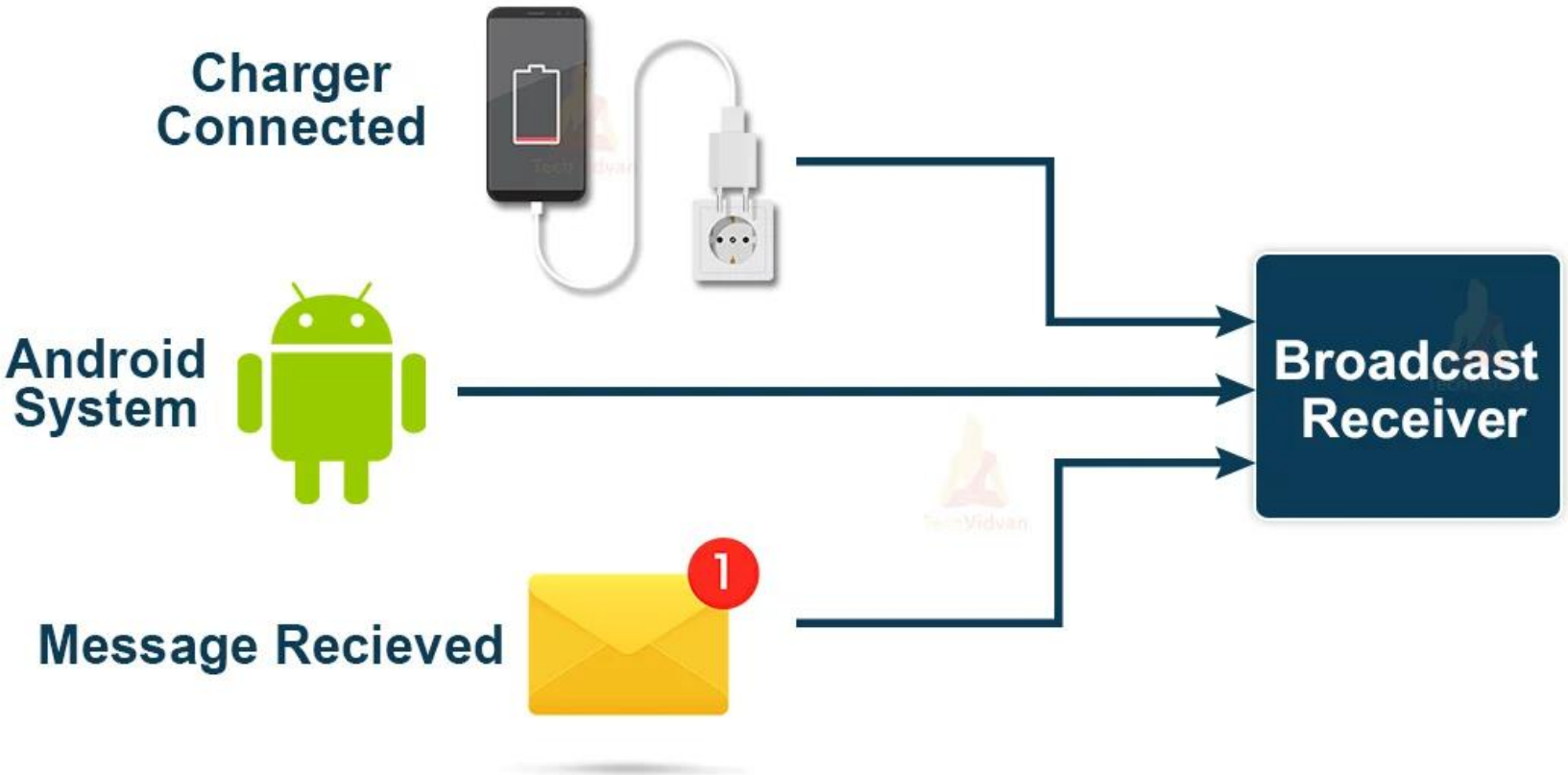
BroadcastReceiver



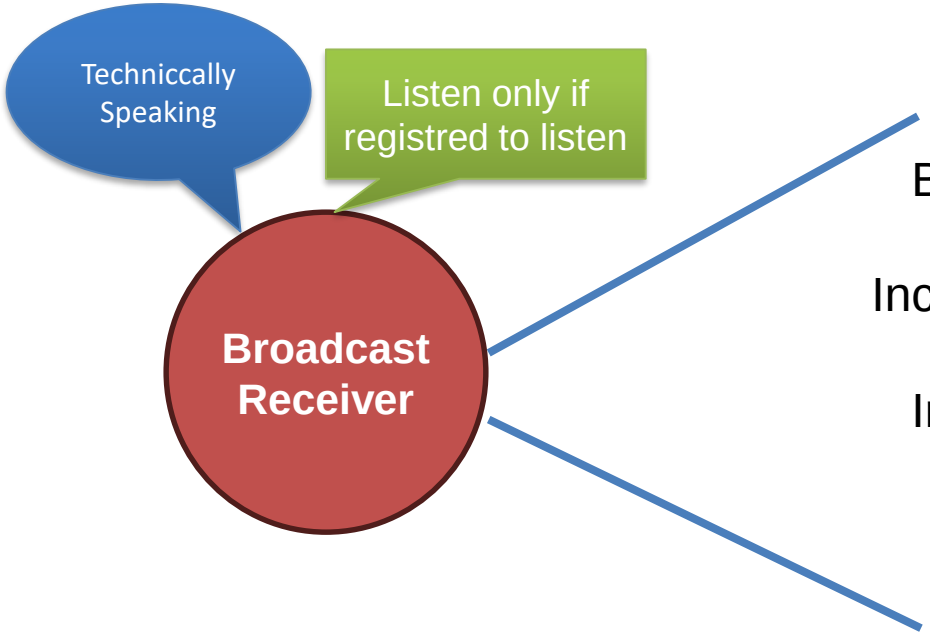
Besoin d'un « Listner »

- Recevoir & réagir aux broadcast intents
- Répondre au message diffusé
- Pas d'interface utilisateur mais peut démarrer une activité
- Répondre aux notifications ou aux changements de statut

BroadcastReceiver



BroadcastReceiver



Ce n'est pas une liste d'événements exhaustive.

Battery Low

Incomming Call

Incoming SMS

Wi-Fi availability

Bluetooth device connected

Charger (dis)connected

BroadcastReceiver: Implementation

A broadcasr receiver est implémenté en tant que sous-classe de la classe **BroadcastReceiver** et chaque message est diffusé en tant qu'objet Intent (**intent object**).

BroadcastReceiver: Implémentation

java

```java
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        // Votre code à exécuter lorsque le broadcast est reçu
    }
}
```

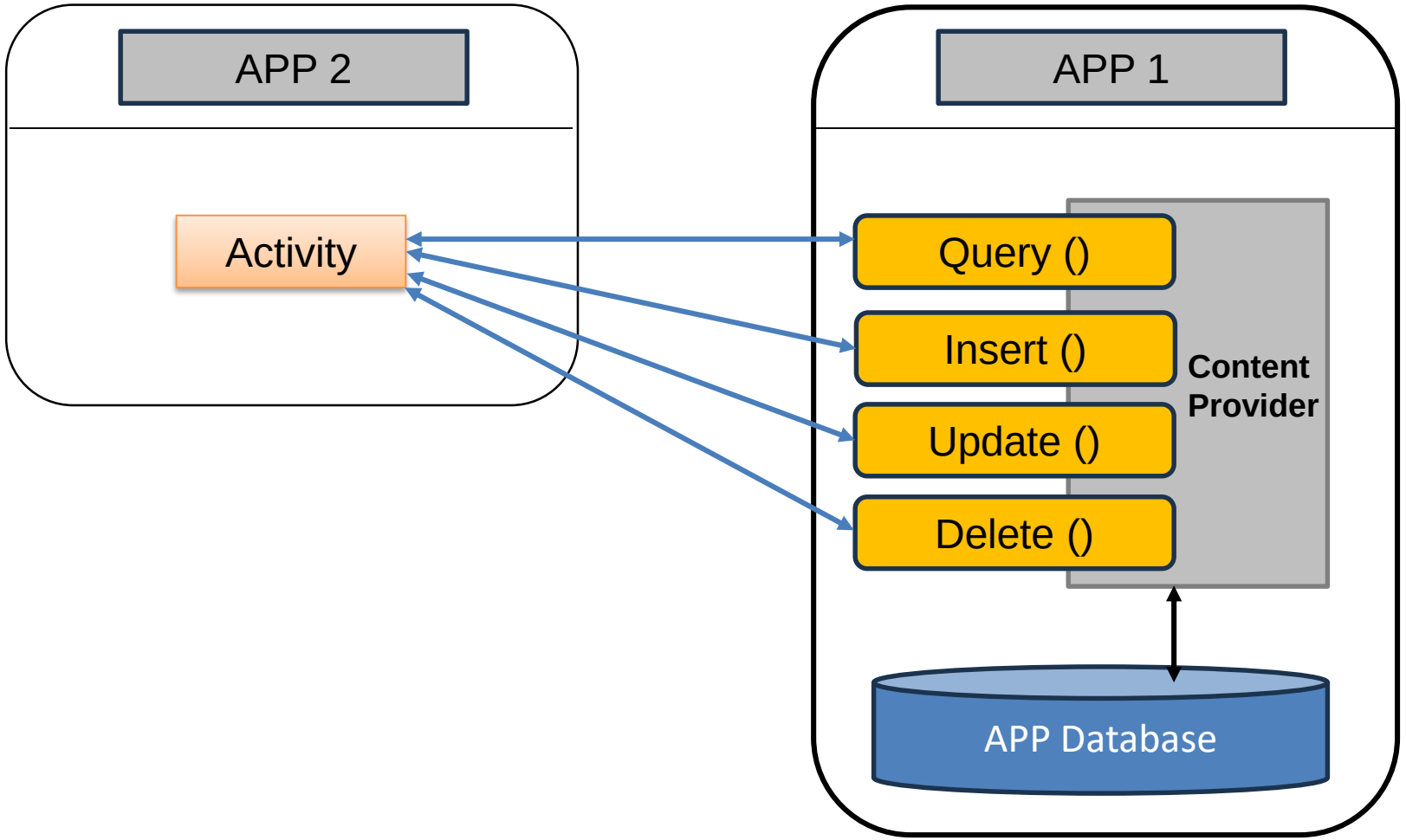
kotlin

```kotlin
class MyReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        // Votre code à exécuter lorsque le broadcast est reçu
    }
}
```

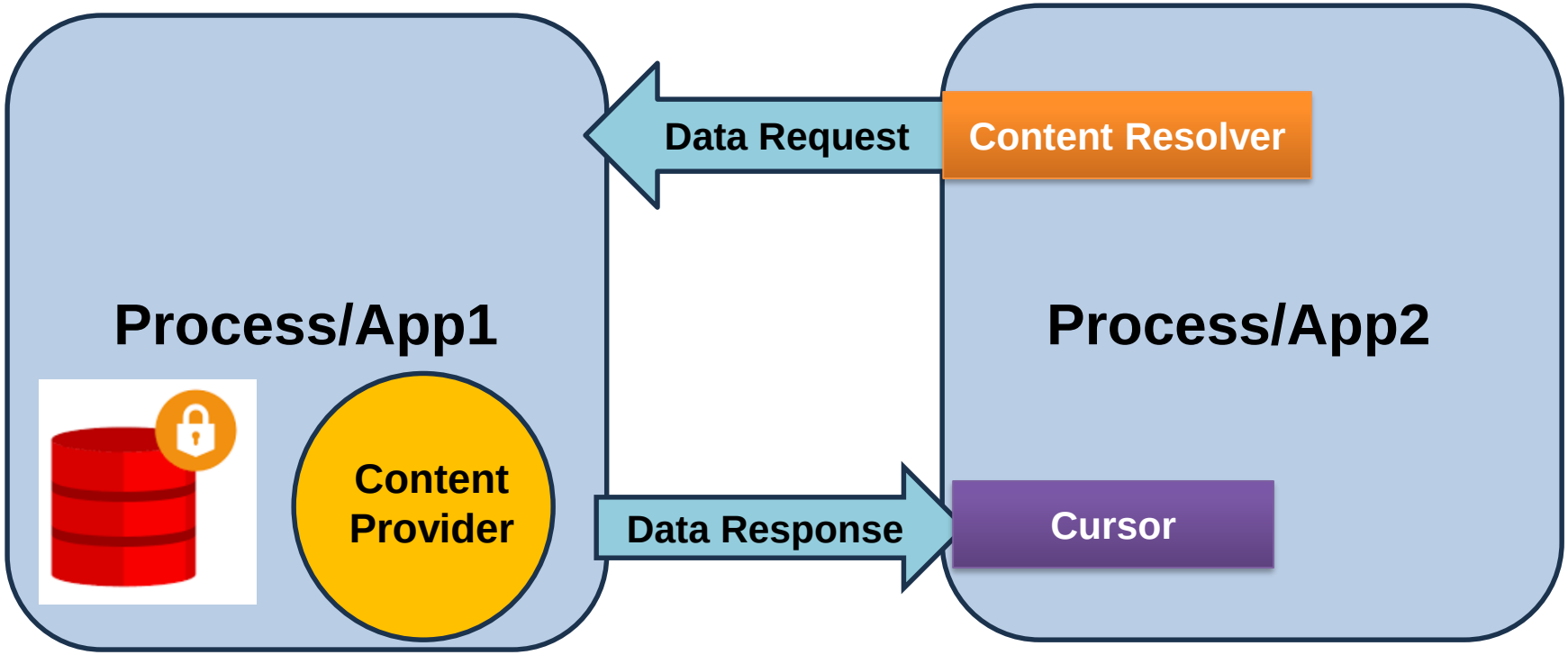
ContentProvider

- Gérer de partager des données de **manière sécurisée** entre différentes applications.
  - Permettant un accès sécurisé aux données via des autorisations
  - Garantissant l'intégrité des données.
- Une passerelle entre l'application et un stockage de données centralisé(par ex. BD SQLite, un fichier texte, un fichier JSON).
- Ils offrent un **mécanisme standardisé** pour effectuer des opérations CRUD (Create, Read, Update, Delete) sur les données,

ContentProvider



ContentProvider



Android OS

ContentProvider

- **Implémentation**

java

```java
public class MyContentProvider extends ContentProvider {
    @Override
    public boolean onCreate() {
        // Votre code à exécuter lors de la création du ContentProvider
        return true;
    }

    // Reste de l'implémentation des méthodes de ContentProvider
}
```

kotlin

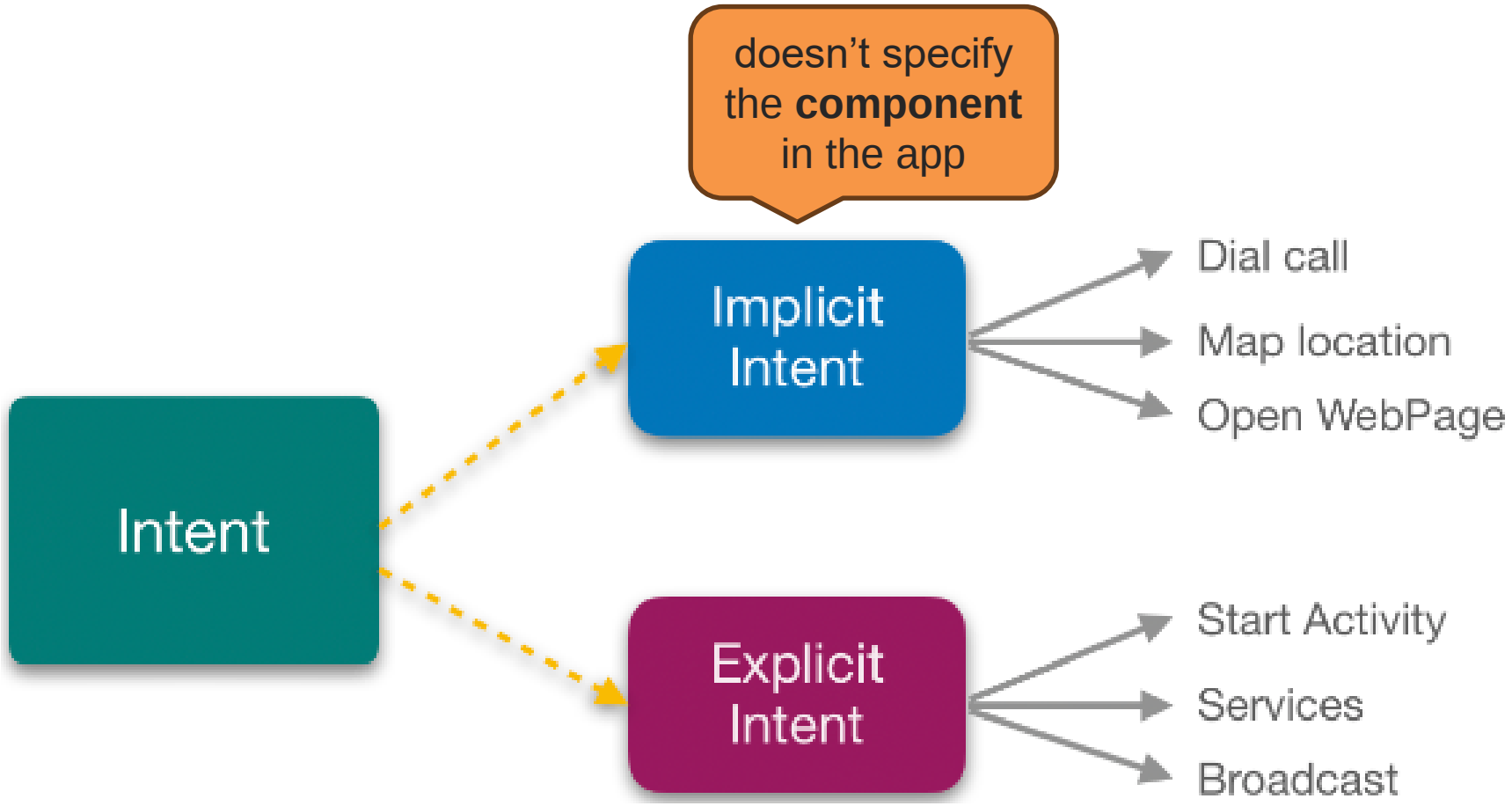
```kotlin
class MyContentProvider : ContentProvider() {
    override fun onCreate(): Boolean {
        // Votre code à exécuter lors de la création du ContentProvider
        return true
    }

    // Reste de l'implémentation des méthodes de ContentProvider
}
```

Intent

- Message qui exprime une intention (« ouvrir cet écran », « démarrer ce service », « signaler cet événement »).
- **Intent** (Intention) sont des messages asynchrones qui permettent aux composants d'application de demander des fonctionnalités à partir d'autres composants **Android**.
- **Intent** est le moyen de transmission entre les composants (activities, content providers, broadcast receivers, services, etc., etc.)
- **intent** est un objet de la classe **android.content.Intent**.

Intent


doesn't specify the **component** in the app
Intent
Implicit Intent
Explicit Intent
Dial call
Map location
Open WebPage
Start Activity
Services
Broadcast

Intent

Identifiez le type de intent dans cette application Android



Intent explicite vs implicite

1. Explicite

On nomme la classe exacte à ouvrir · Dans sa propre application

```
// Bouton « Ouvrir mon profil »
btnProfil.setOnClickListener {
  val intent = Intent(this,
      ProfileActivity::class.java)
  startActivity(intent)
}
```

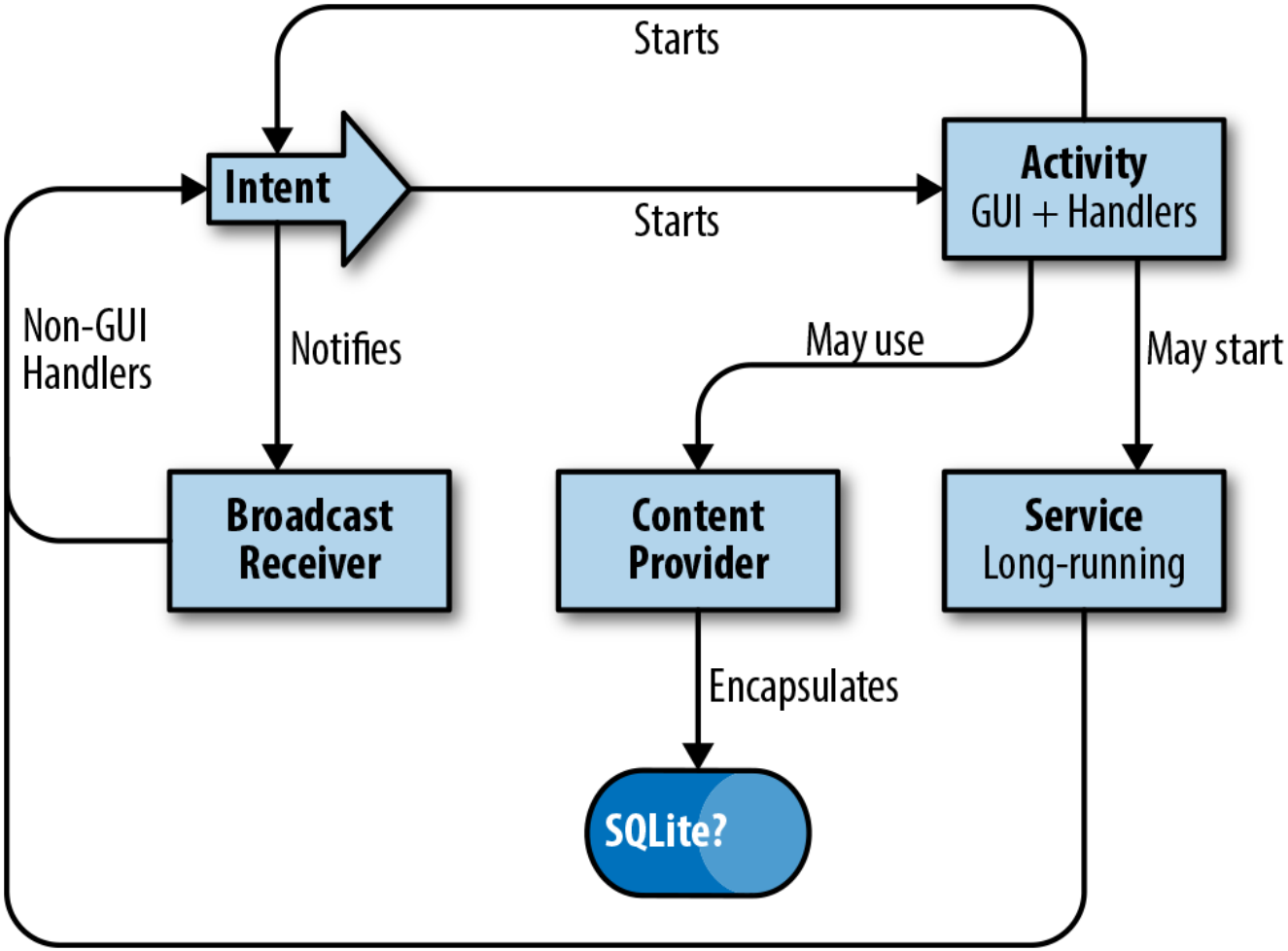
2. Implicite

On décrit l'action, Android choisit l'app · Vers une autre application

```
// Bouton « Afficher ma position »
btnMaps.setOnClickListener {
  val uri = Uri.parse(
      "geo:48.8584,2.2945")
  val intent = Intent(
      Intent.ACTION_VIEW, uri)
// aucune classe nommée → implicite
  startActivity(intent)
}
```

À retenir : Explicite = « Ouvre cet écran précis ». Implicite = « Je veux cette action, trouve-moi une application ».

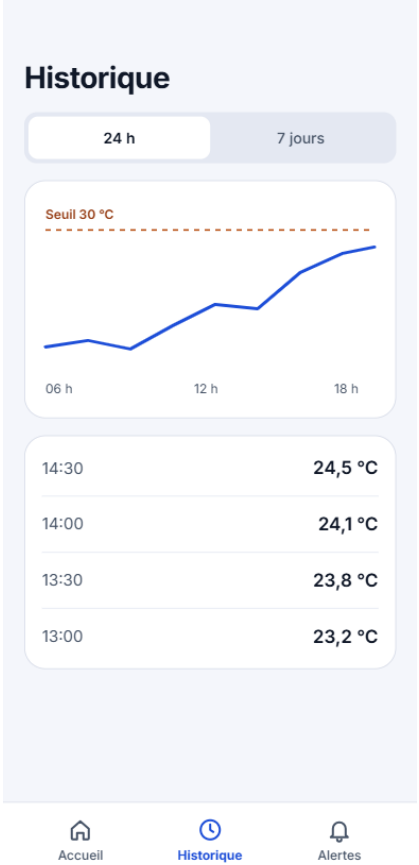
Android Activity Lifecycle

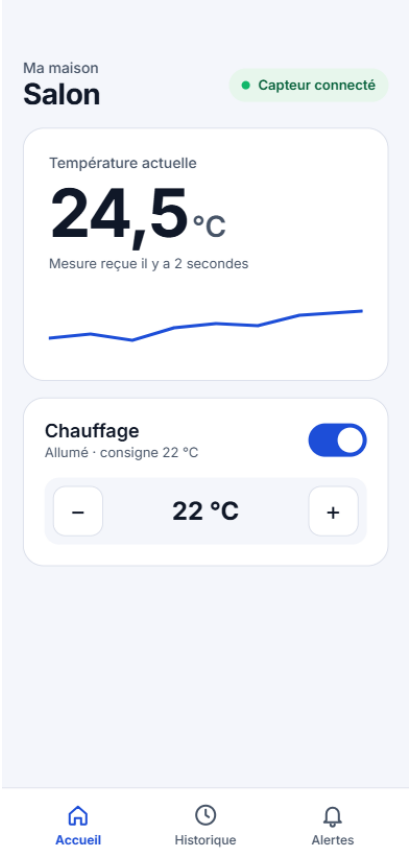


Exercice : application « SmartHome »

Scénario: Une application Android contrôle un capteur de température IoT (ESP32) installé dans une maison. Elle doit répondre aux 4 besoins du tableau ci-dessous. Complétez le tableau : pour chaque besoin, indiquez le composant Android adapté et justifiez votre choix en une phrase.







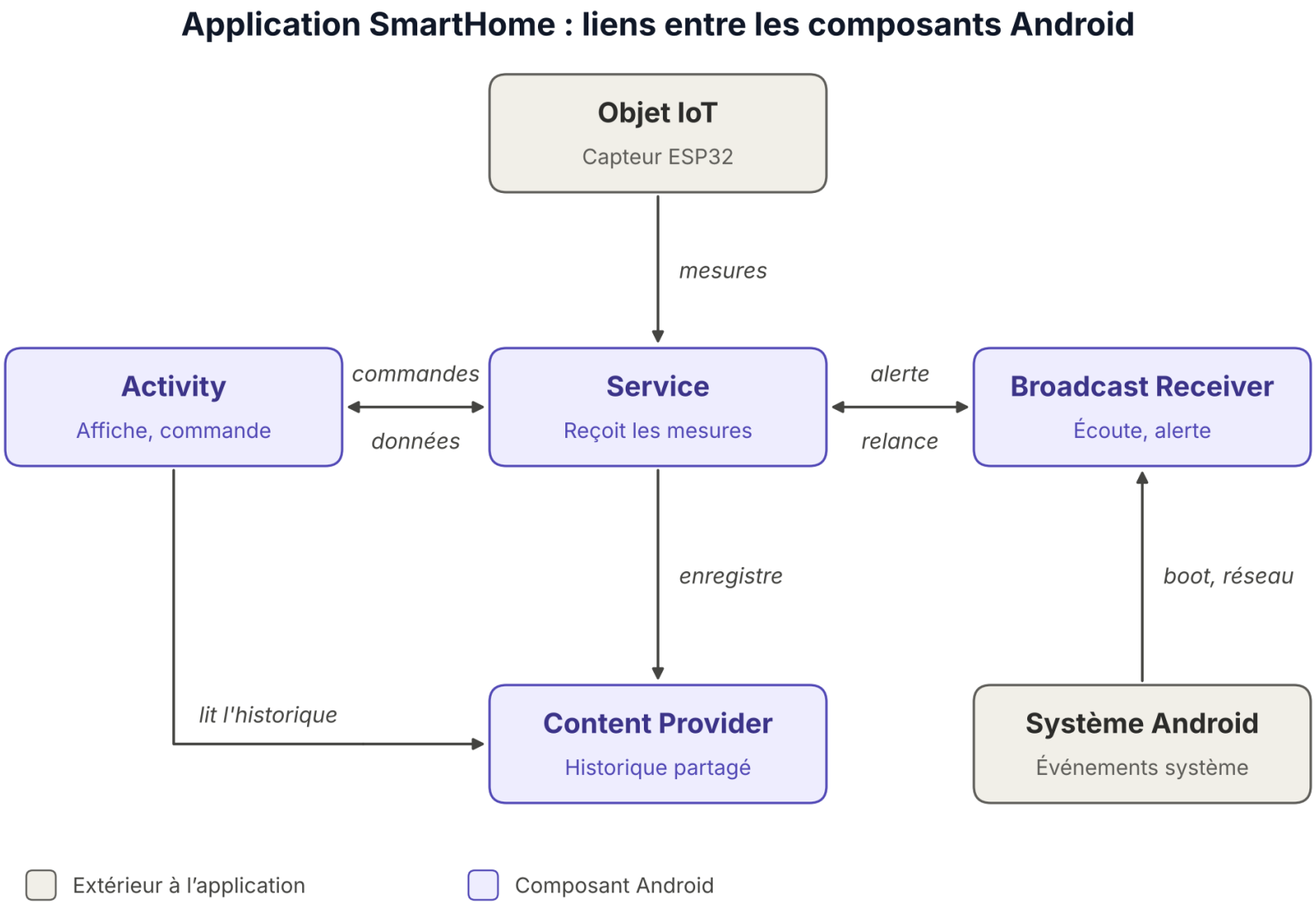
Exercice : application « SmartHome »

Scénario
Une application Android contrôle un capteur de température IoT (ESP32) installé dans une maison. Elle doit répondre aux 4 besoins du tableau ci-dessous.

Travail demandé
Complétez le tableau : pour chaque besoin, indiquez le composant Android adapté et justifiez votre choix en une phrase.

Besoin	Composant	Pourquoi
Afficher la température et le bouton chauffage		
Recevoir les mesures en continu, même écran fermé		
Redémarrer l'application après le boot du téléphone et alerter si la température dépasse 30 °C		
Conserver l'historique des mesures, lisible par d'autres applications		

Exercice : application « SmartHome »



Exercice : application «Smart Farm »

Scénario

Une exploitation agricole utilise des **ESP32** (Wi-Fi/MQTT) : capteurs de température/humidité (serre), humidité du sol, niveau d'eau, plus une **pompe** et un **ventilateur** pilotables. Une application Android permet de surveiller et contrôler l'ensemble.

N°	Besoin	Composant	Justification
1	Afficher en temps réel températures, humidité et niveau d'eau		
2	Garder la connexion MQTT en permanence, même app fermée, avec notification persistante		
3	Détecter la perte de Wi-Fi et passer en « mode hors ligne »		
4	Stocker l'historique SQLite et le partager avec un widget et une application tierce		
5	Synchroniser avec le cloud chaque heure, en Wi-Fi seulement et hors économie de batterie		
6	Relancer la surveillance après redémarrage du téléphone		
7	Alerte si température > 35 °C, avec bouton « Activer le ventilateur » dans la notification		
8	Passer du tableau de bord au détail d'une parcelle en transmettant son identifiant		
9	Appeler le technicien en un clic depuis la fiche d'une panne		
10	Donner à l'écran de contrôle un accès direct à startPump(), stopPump(), getStatus()		

Composants supplémentaires

Component	Description
Fragments	Represents a portion of the user interface in an Activity.
Views	UI elements that are drawn on-screen including buttons, lists, forms, etc.
Layouts	View hierarchies that control screen format and appearance of the views.
Intents	Messages wiring components together.
Resources	External elements, such as strings, constants, and drawable pictures.
Manifest	Configuration file for the application.

TP Android : Broadcast Receiver, Service et Content Provider

Objectif : programmer trois composants Android et les tester sur l'émulateur ou un téléphone.



Broadcast Receiver
Branchement de la batterie

- Détecter le branchement et le débranchement du chargeur



Service
Caméra et batterie

- Créer SurveillanceService qui étend Service
- Lire le niveau de la batterie avec BatteryManager
- Prendre une photo avec la caméra si la batterie est faible ou si le chargeur est débranché



Content Provider
Contacts du téléphone

- Lire les contacts avec un ContentResolver et ContactsContract

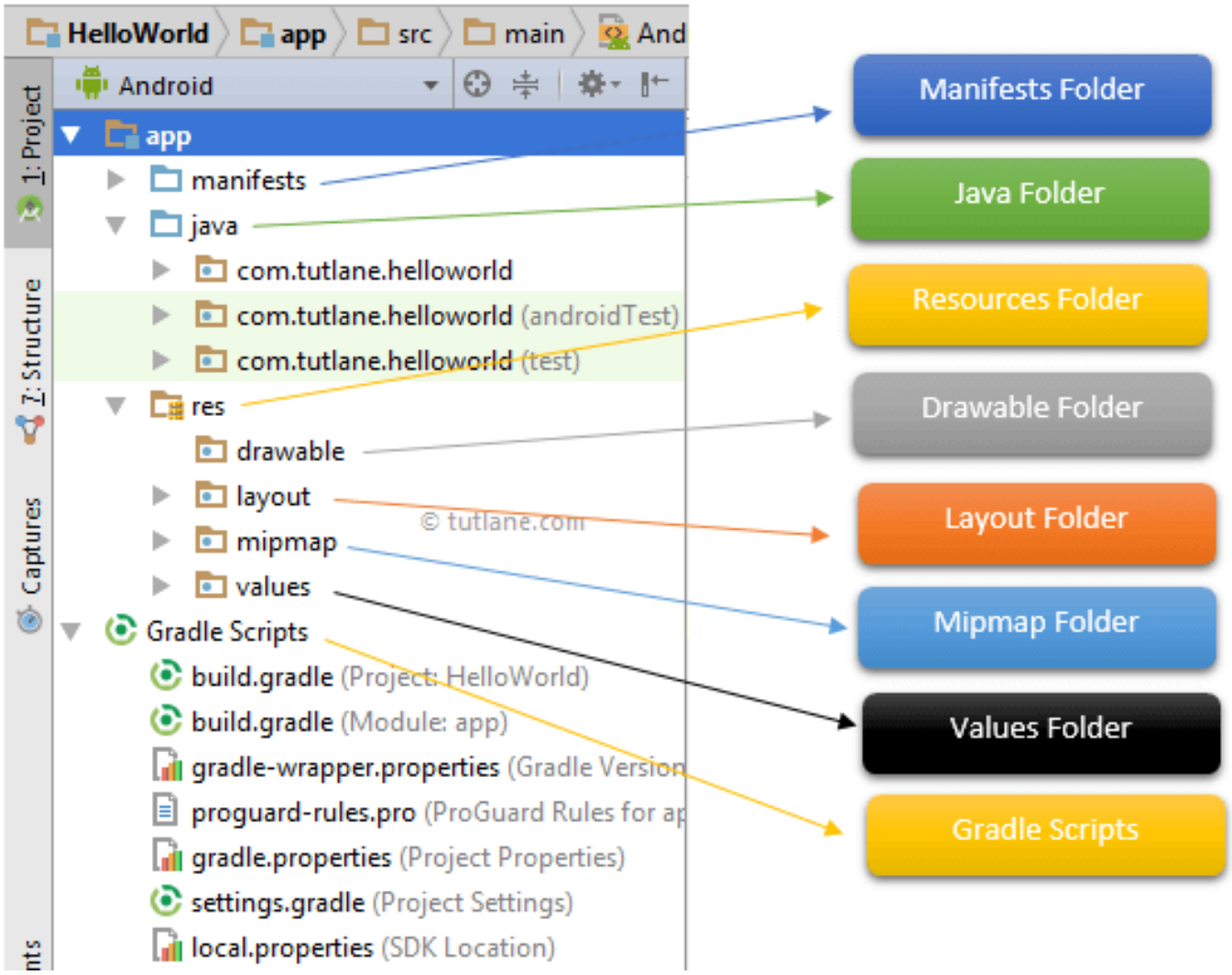
Scénario : chargeur débranché → le Broadcast Receiver démarre le Service → le Service prend une photo.

Livrable : une application avec trois boutons (Batterie, Caméra, Contacts) et le Manifest complété.

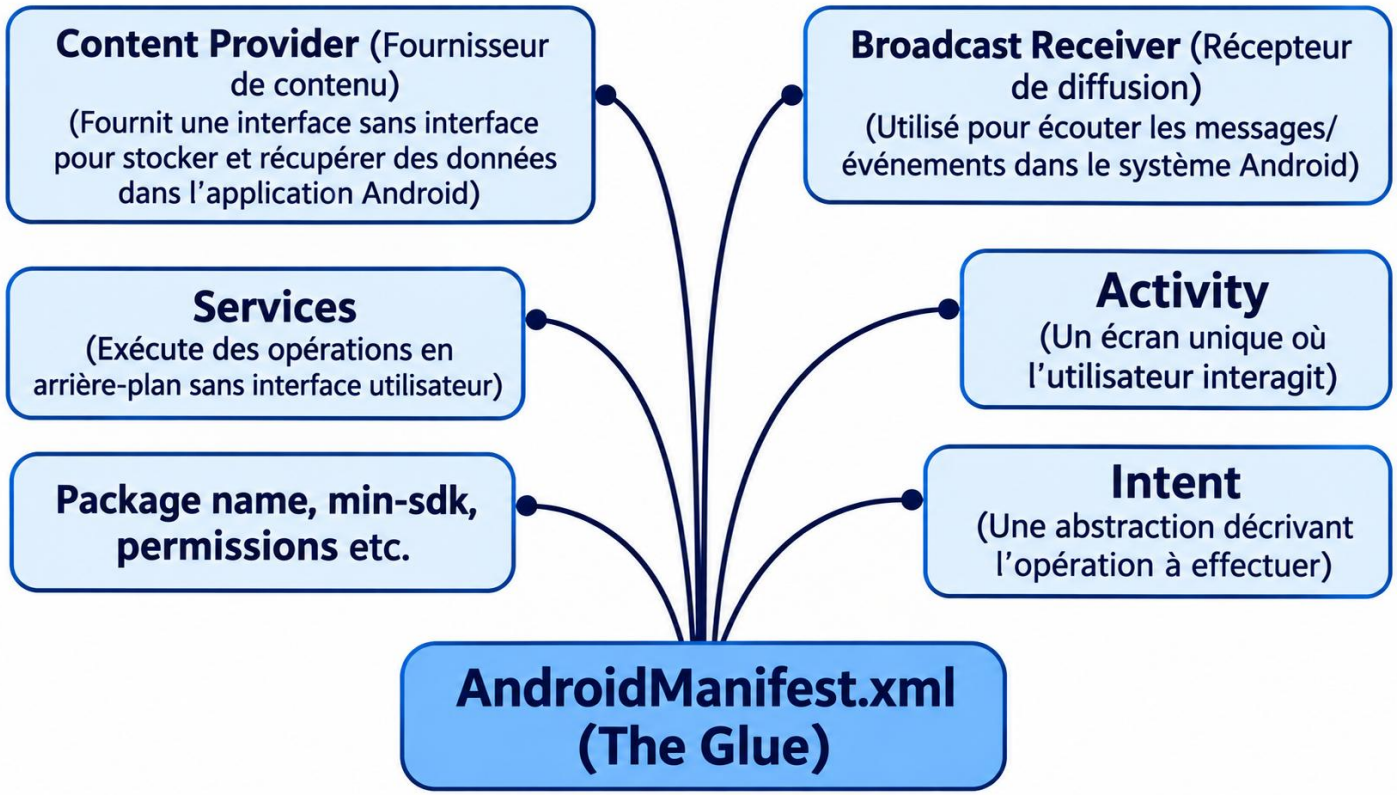
Structure de projet d'une application Android



Structure de projet d'une application Android



AndroidManifest



AndroidManifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
          package="com.example"
          android:versionCode="1"
          android:versionName="1.0">

    <uses-sdk android:minSdkVersion="15"/>

    <application android:label="@string/app_name"
                 android:debuggable="true"
                 android:icon="@drawable/ic_launcher" />

        <activity android:name="MainActivity"
                  android:label="@string/app_name">
          <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
          </intent-filter>
        </activity>

        <service>...</service>

        <receiver>...</receiver>

        <provider>...</provider>

    </application>
</manifest>
```

1. Manifest element
2. Uses-sdk element
3. Application element
4. Activity element
5. Service element
6. Receiver element
7. Provider element

The AndroidManifest.xml file

Permissions: The <uses-permission> element is used to declare permissions that the application needs to access certain features or data on the device. For example, accessing the internet, reading contacts, or accessing the camera.

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

The AndroidManifest.xml file

Access to Application and User Data: The <application> element contains various attributes and child elements that define how the application interacts with the user and the system. This includes specifying the app's icon, label, theme, activities, services, broadcast receivers, content providers, and more

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme">

    <!-- Activities, services, broadcast receivers, content providers, etc.

</application>
```

The AndroidManifest.xml file

Declaration of Components: Activities, services, broadcast receivers, and content providers are declared within the <application> element. For example, an <activity> element declares an activity component:

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

The AndroidManifest.xml file

Content Providers: Content providers, if your application uses them, are declared in the manifest. Content providers allow your application to share data with other applications. For example:

xml Copy code

```xml
<provider
    android:name=".MyContentProvider"
    android:authorities="com.example.myapp.provider"
    android:exported="true">
</provider>
```

The AndroidManifest.xml file

To add support for right-to-left (RTL) languages such as Arabic in your Android app, you need to make certain adjustments in your AndroidManifest.xml file as well as in your layout files.

xml

```xml
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:supportsRtl="true"
    android:theme="@style/AppTheme">
    <!-- Other application attributes -->
</application>
```

Thank you !

Questions ? abdelkader.ouared@univ-tiaret.dz



www. .univ-tiaret.dz